

Course

« *Deep Learning* »

Sid-Ahmed Berrani

ENSIA

2024 – 2025

3. Deep Neural Network

Regularization: Using Data Augmentation

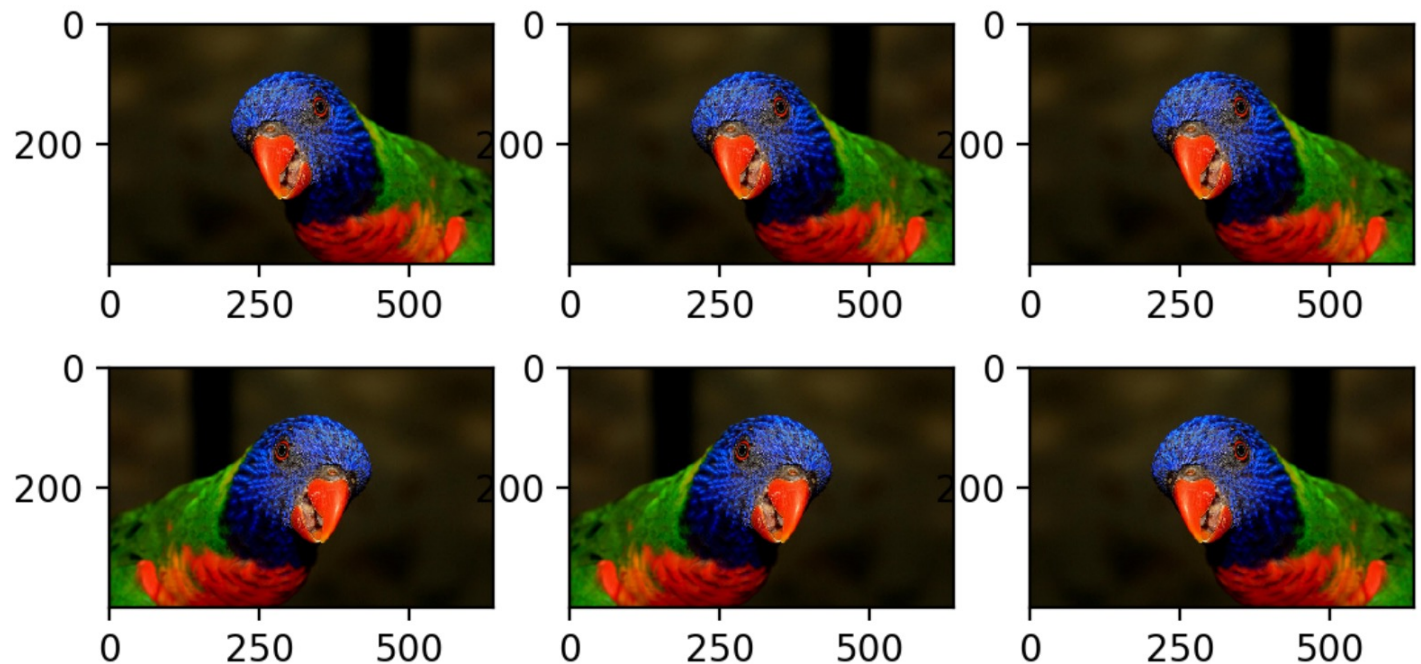
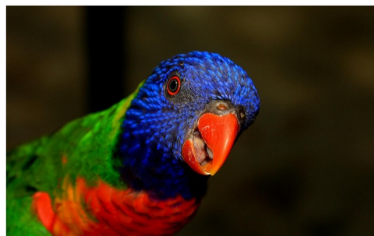
- A possible solution to overfitting: More training data.
- Unfortunately, not always possible in practice.
- An alternative: Generate new examples from existing ones.
- Particularly adapted to computer vision applications.
- Principle: Artificially expand the size of a training dataset by creating modified versions of images in the train dataset.

Ref. : <https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html>

3. Deep Neural Network

Regularization: Data Augmentation

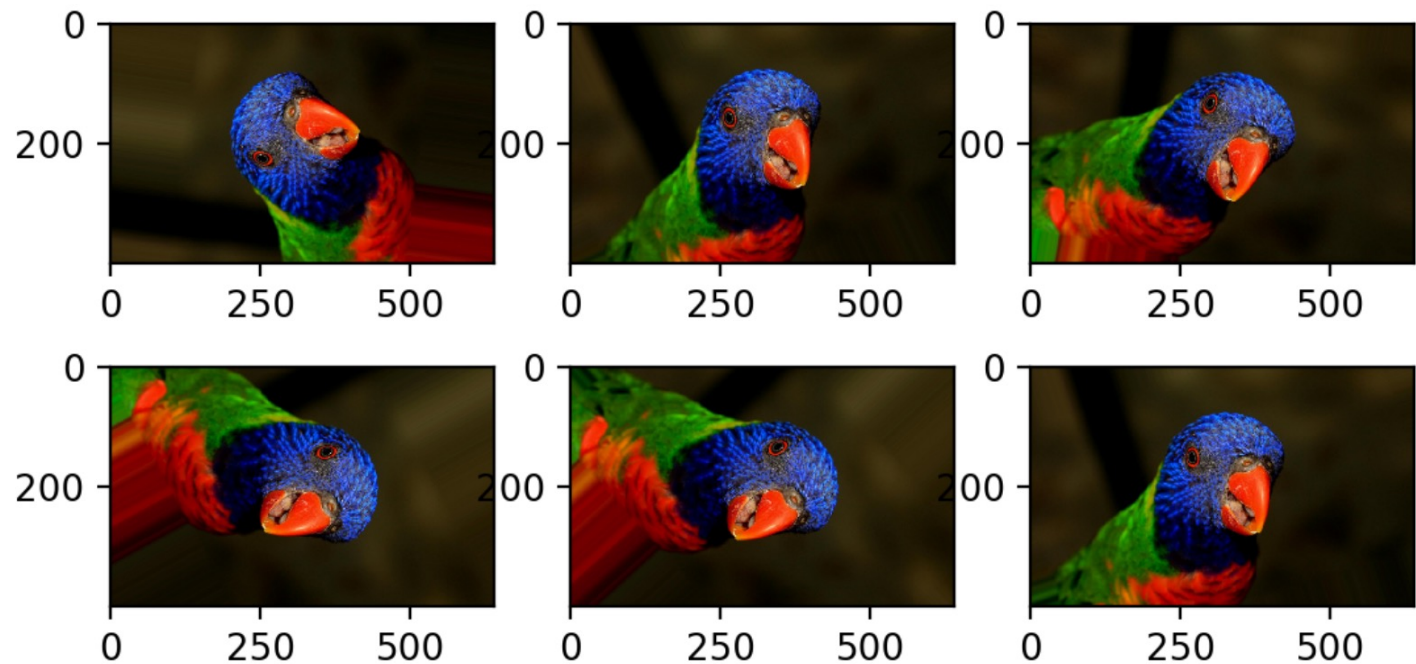
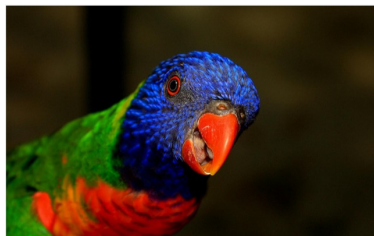
Horizontal and Vertical Flip Augmentation



3. Deep Neural Network

Regularization: Data Augmentation

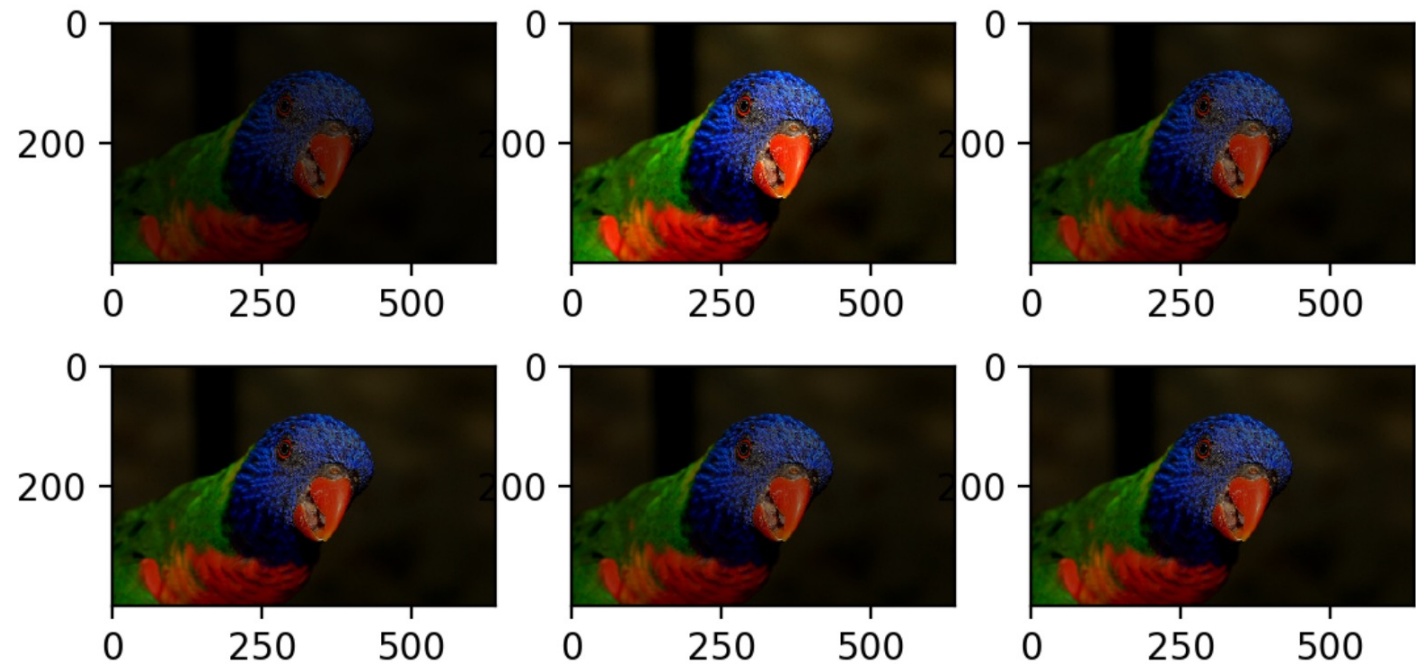
Random Rotation Augmentation



3. Deep Neural Network

Regularization: Data Augmentation

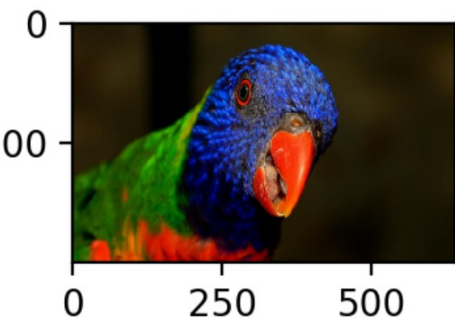
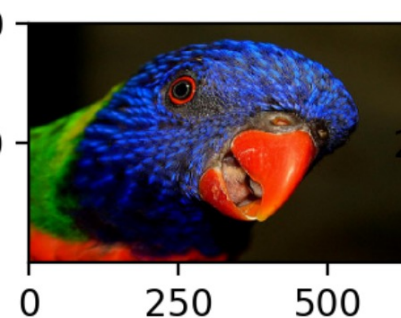
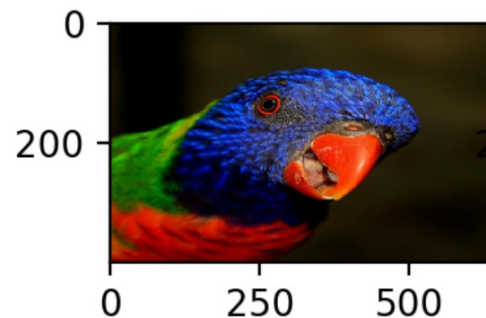
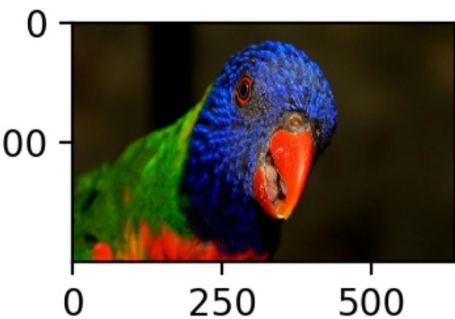
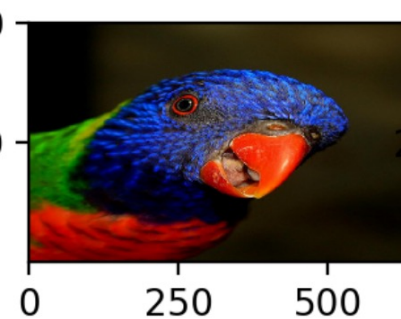
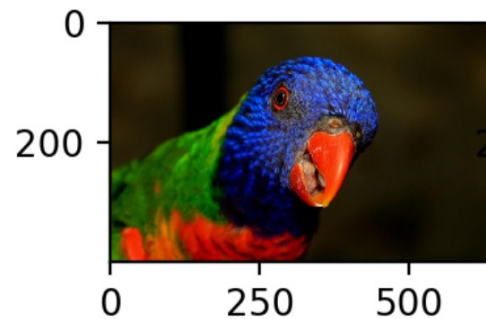
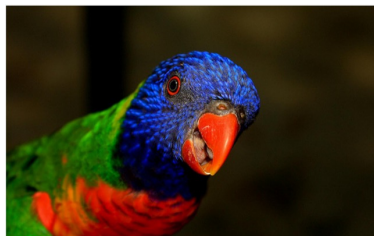
Random Brightness Augmentation



3. Deep Neural Network

Regularization: Data Augmentation

Random Zoom Augmentation



3. Deep Neural Network

Regularization: Data augmentation

- Using Keras, data augmentation can be done the class: `keras.preprocessing.image.ImageDataGenerator`

```
from keras.preprocessing.image import ImageDataGenerator, array_to_img, img_to_array, load_img

datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest')

img = load_img('data/train/cats/cat.0.jpg') # this is a PIL image
x = img_to_array(img) # this is a Numpy array with shape (3, 150, 150)
x = x.reshape((1,) + x.shape) # this is a Numpy array with shape (1, 3, 150, 150)

# the .flow() command below generates batches of randomly transformed images
# and saves the results to the `preview/` directory
i = 0
for batch in datagen.flow(x, batch_size=1,
                          save_to_dir='preview', save_prefix='cat', save_format='jpeg'):
    i += 1
    if i > 20:
        break # otherwise the generator would loop indefinitely
```

3. Deep Neural Network

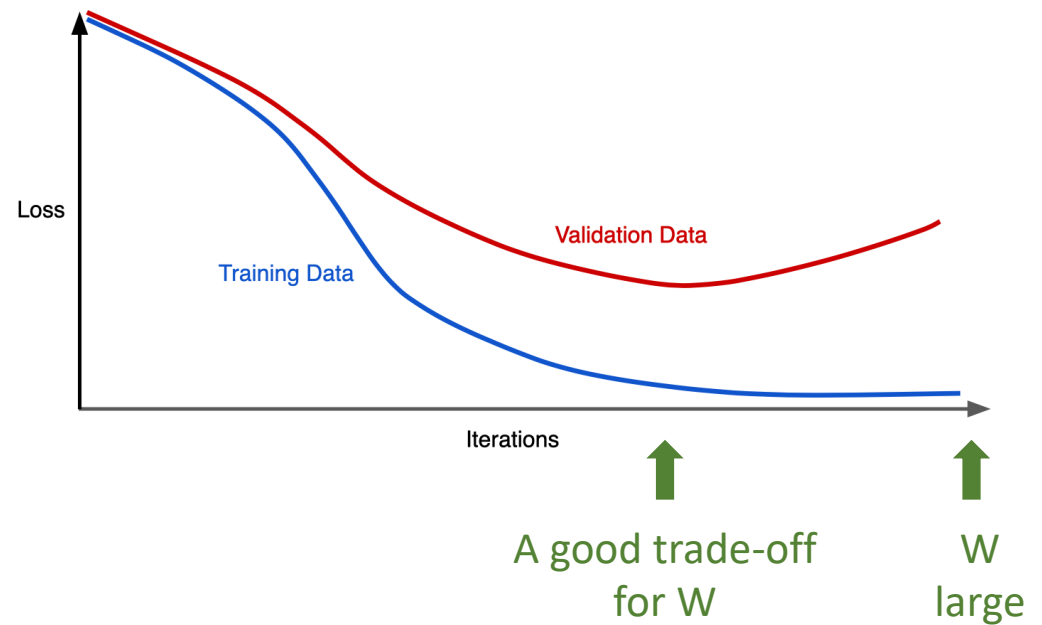
Regularization: Data augmentation

- Using Keras, data augmentation can be done the class: `keras.preprocessing.image.ImageDataGenerator`



3. Deep Neural Network

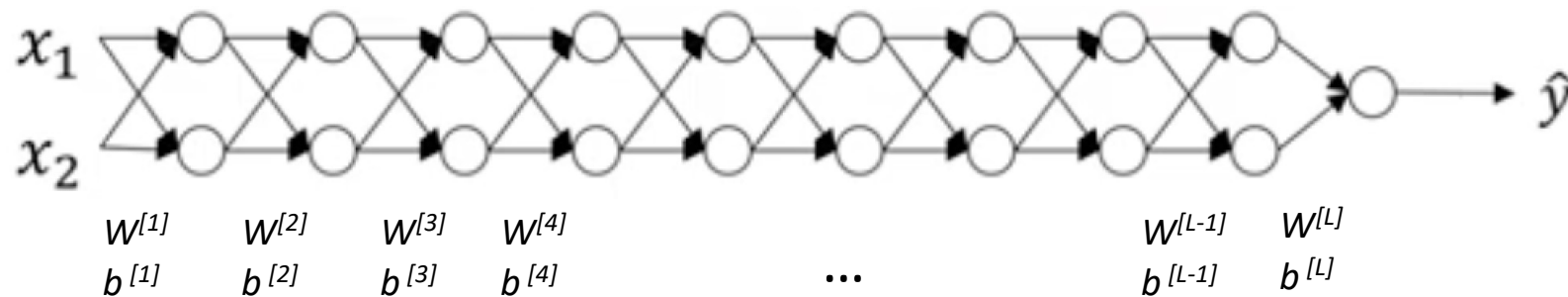
Regularization: Using Early Stopping



- Remember that L2 regularization also controls weights W .

3. Deep Neural Network

Vanishing and exploding gradient



Let's consider:

$$b^{[i]} = 0$$

$g(z) = z$ (linear activation function)

$$W^{[i]} = \begin{bmatrix} p & 0 \\ 0 & p \end{bmatrix}$$

$$\hat{y} = W^{[L]} \cdot W^{[L-1]} \dots W^{[2]} \cdot \boxed{W^{[1]} \cdot x}$$

$$Z^{[1]} = W^{[1]} \cdot x$$

$$A^{[1]} = g(Z^{[1]}) = Z^{[1]}$$

$$\hat{y} = \begin{bmatrix} p & 0 \\ 0 & p \end{bmatrix}^{[L]} \cdot x = \begin{bmatrix} p^L & 0 \\ 0 & p^L \end{bmatrix} \cdot x$$

3. Deep Neural Network

Vanishing and exploding gradient

One partial solution:

Weight initialization.

In our labs:

For logistic regression, weights have been initialized to 0.

For feed-forward NN, weights have been initialized randomly.

Why?

3. Deep Neural Network

Vanishing and exploding gradient

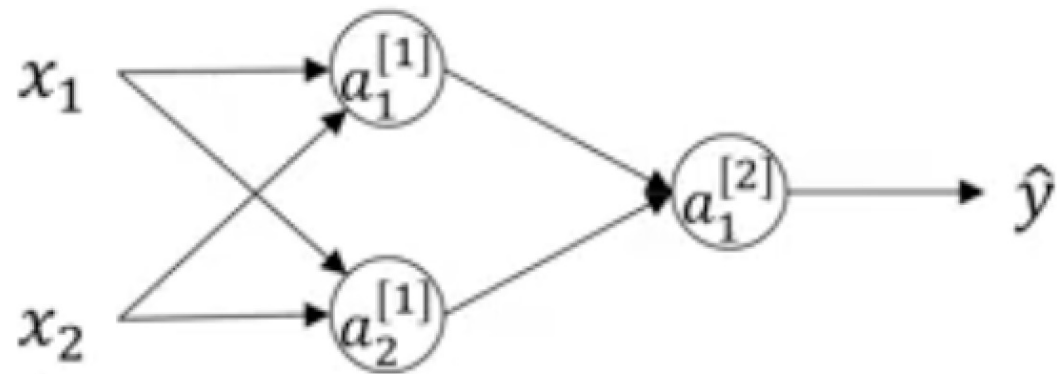
One partial solution:
Weight initialization.

Symmetry:

The 2 hidden units are computing exactly the same function

No need to have more than one hidden unit!

⇒ **Solution:** Initialize weights randomly.



$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$a_1^{[1]} = a_2^{[1]}$$

$$\frac{dJ}{dz_1^{[1]}} = \frac{dJ}{dz_2^{[1]}}$$

$$\frac{dJ}{dW^{[1]}} = \begin{bmatrix} u_1 & u_2 \\ u_1 & u_2 \end{bmatrix}$$

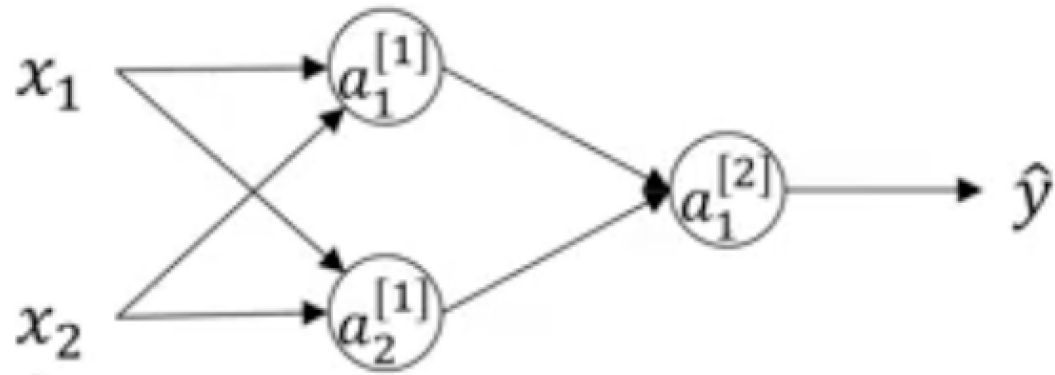
$$W^{[1]} = W^{[1]} - \alpha \cdot \frac{dJ}{dW^{[1]}}$$

3. Deep Neural Network

Weight random initialization

Weight initialization.

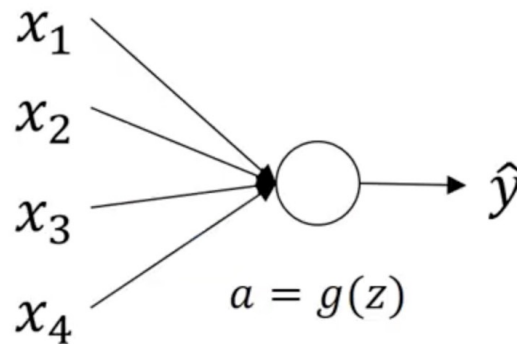
$W^{[1]} = np.random.randn((2,2)) \cdot 0.01$
 $b^{[1]} = np.zeros((2,2))$



Usually, we prefer to initialize weights to very small values (cf. last week's lecture).

3. Deep Neural Network

Weight initialization for deep networks



$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + \cancel{b}$$

z not too large $\Rightarrow$ The larger n , the smaller w_i

We can notice that:

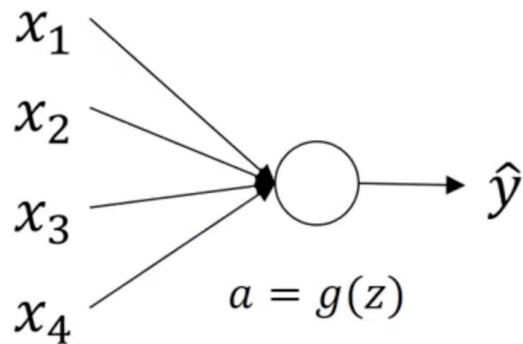
The variance of z linearly increases as the number of inputs increases.

Variance of the sum of independent variables = Sum of variances.

$\text{variance}(A+B) = \text{variance}(A) + \text{variance}(B)$, if A and B are independent

3. Deep Neural Network

Weight initialization for deep networks

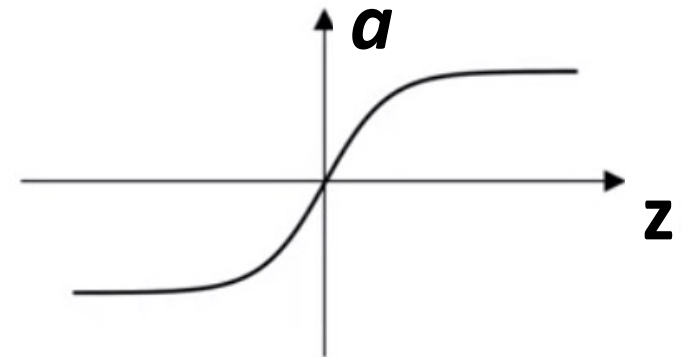


$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

For the tanh activation function, we target a variance of $z = 1$ (cf. last week's lecture).

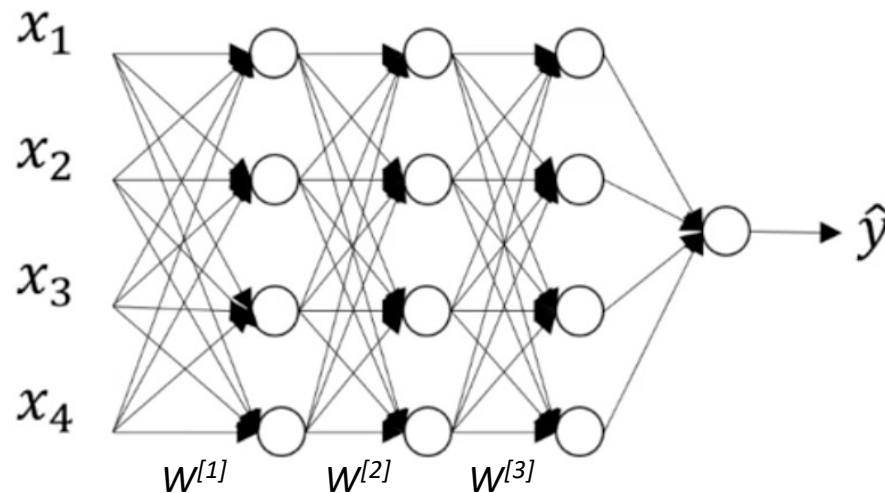
$$\Rightarrow \text{variance}(w_i) = 1/n$$

$$W^{[1]} = np.random.randn((4,1)) * np.sqrt{\frac{1}{4}}$$



3. Deep Neural Network

Weight initialization for deep networks



For each layer's weights:

Step 1: Randomly initialize the weights (from a Gaussian distribution or a uniform distribution).

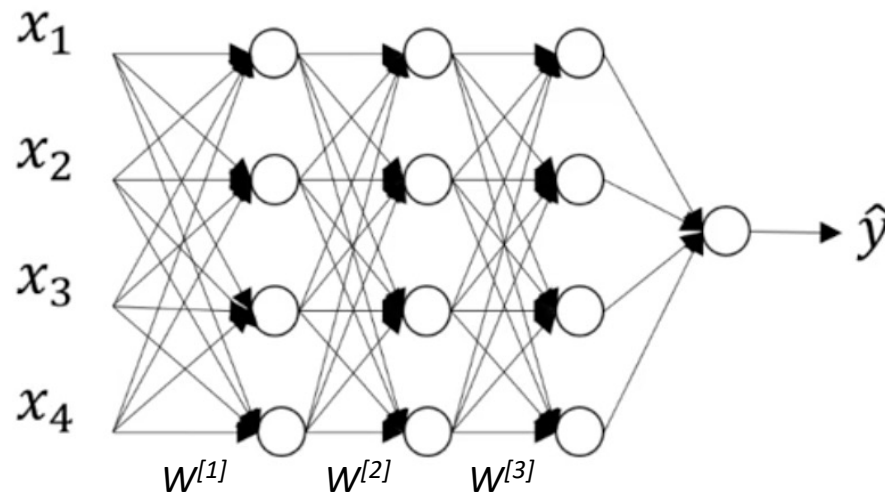
Step 2: Scale the weights proportional to the number of inputs of the layer.

The scale factor:

- For the first hidden layer : the number of input features.
- For the second hidden layer : the number of units in the first hidden layer.
- ...

3. Deep Neural Network

Weight initialization for deep networks



$$W^{[L]} = np.random.randn((n^{[l-1]}, n^{[l]})) * np.sqrt{\frac{1}{n^{[l-1]}}}$$

=> **Xavier initialization**

Xavier Glorot, Yoshua Bengio. **Understanding the difficulty of training deep feedforward neural networks**. Proceedings of the thirteenth international conference on artificial intelligence and statistics. 2010.

Citations = 26 075 (Nov. 17, 2024)

<https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>

3. Deep Neural Network

Weight initialization for deep networks

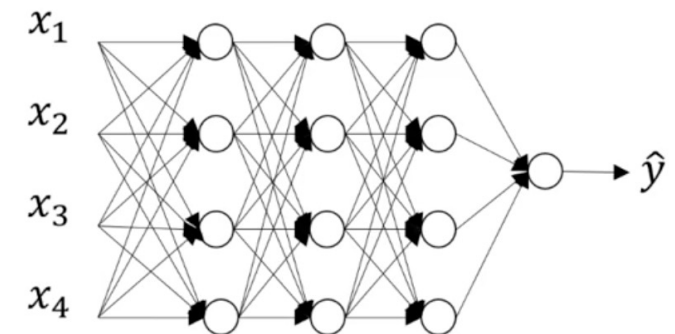
Regarding the bias :

In general, initialized to zero.

If you do not want to initialize it to zero, you can initialize it using Xavier method.

Regarding the variance :

$$\begin{aligned}\text{Var} \left(z_j^{(l)} \right) &= \text{Var} \left(\sum_{k=1}^{m^{(l-1)}} W_{jk}^{(l)} a_k^{(l-1)} \right) \\ &= \sum_{k=1}^{m^{(l-1)}} \text{Var} \left[W_{jk}^{(l)} a_k^{(l-1)} \right] = \sum_{k=1}^{m^{(l-1)}} \text{Var} \left[W_{jk}^{(l)} \right] \text{Var} \left[a_k^{(l-1)} \right] \\ &= \sum_{k=1}^{m^{(l-1)}} \text{Var} \left[W^{(l)} \right] \text{Var} \left[a^{(l-1)} \right] = m^{(l-1)} \text{Var} \left[W^{(l)} \right] \text{Var} \left[a^{(l-1)} \right]\end{aligned}$$



3. Deep Neural Network

Weight initialization for deep networks

NN – Tanh – Without Xavier initialization

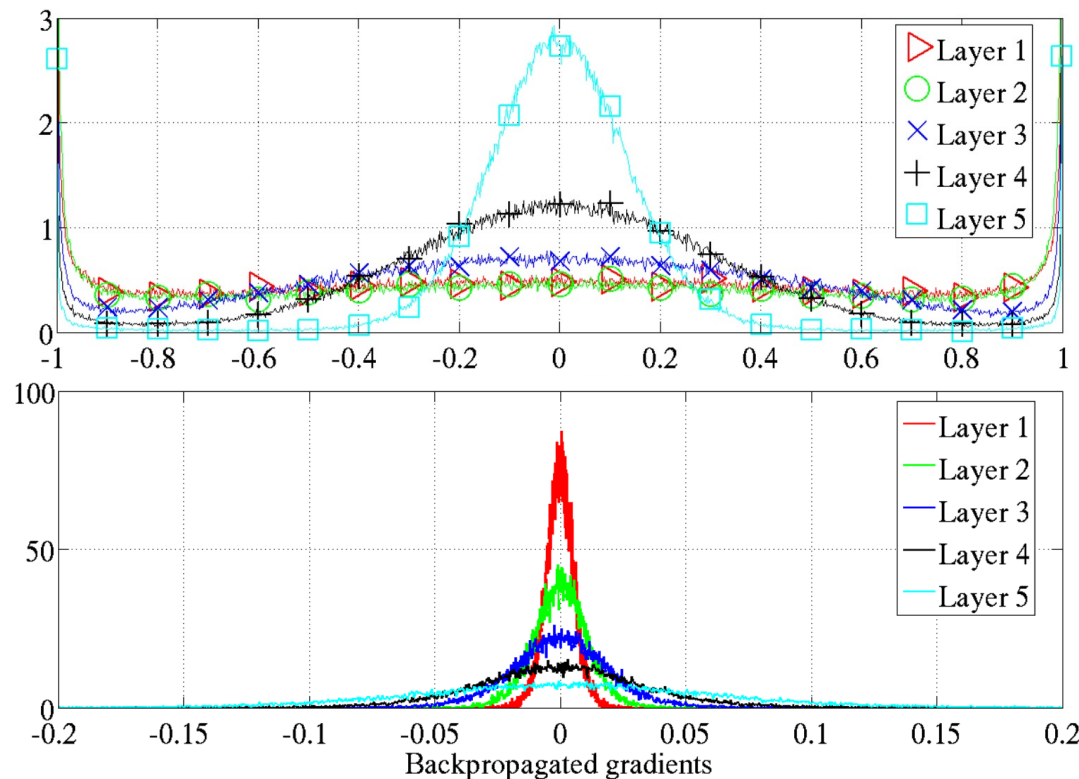


Figure 4: Activation values normalized histogram at the end of learning, averaged across units of the same layer and across 300 test examples. Top: activation function is hyperbolic tangent, we see important saturation of the lower layers.

Figure 7: Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard initialization.

Xavier Glorot, Yoshua Bengio. **Understanding the difficulty of training deep feedforward neural networks**. Proceedings of the thirteenth international conference on artificial intelligence and statistics. 2010.

3. Deep Neural Network

Weight initialization for deep networks

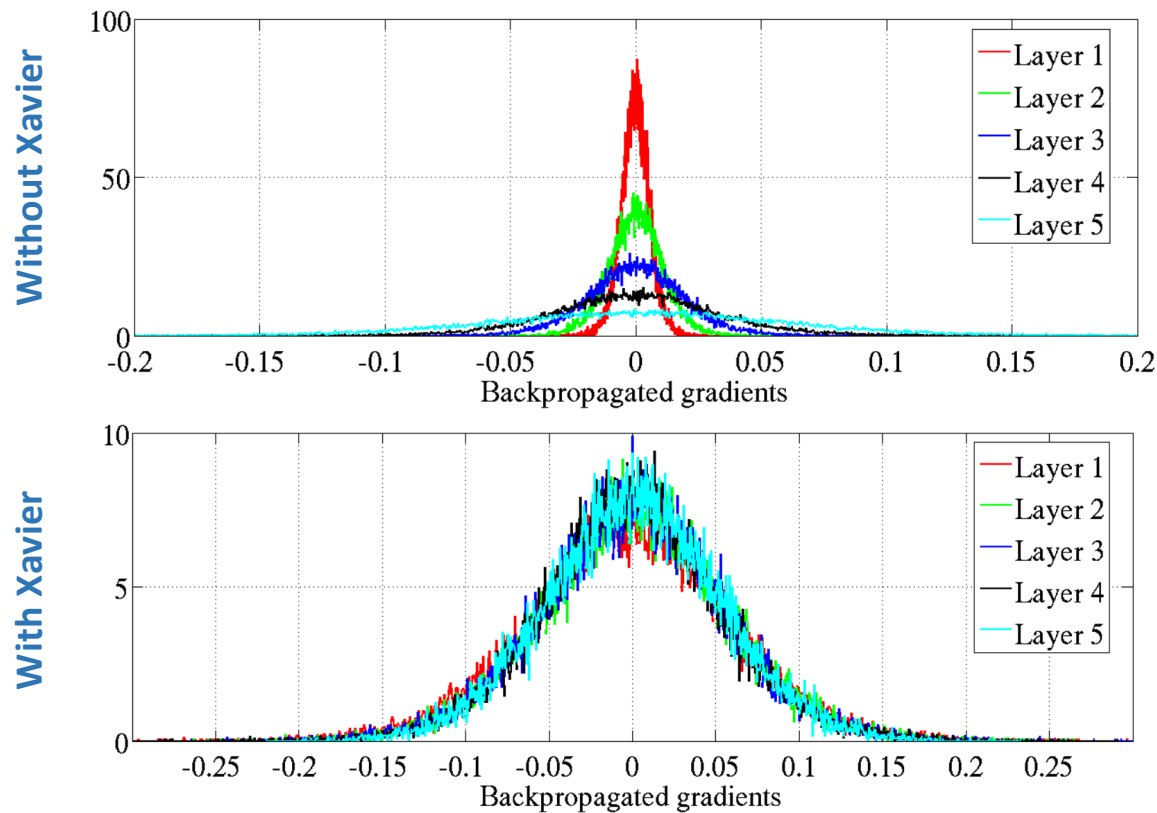


Figure 7: Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization.

Xavier Glorot, Yoshua Bengio. **Understanding the difficulty of training deep feedforward neural networks.** Proceedings of the thirteenth international conference on artificial intelligence and statistics. 2010.

3. Deep Neural Network

Weight initialization for deep networks

Another initialization method: The HE initialization

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV 2015. [# citations = 25988] (Nov. 17, 2014)

<https://arxiv.org/abs/1502.01852>

=> For ReLU activations.

$$\text{Var}[y_l] = \frac{1}{2} n_l \text{Var}[w_l] \text{Var}[y_{l-1}].$$

$$\frac{1}{2} n_l \text{Var}[w_l] = 1, \quad \forall l.$$

$$W^{[L]} = \text{np.random.randn}\left((n^{[l-1]}, n^{[l]})\right) * \text{np.sqrt}\left(\frac{2}{n^{[l-1]}}\right)$$

3. Deep Neural Network

Weight initialization for deep networks

The HE initialization

Figure 2. The convergence of a 22-layer large model (B in Table 3). The x-axis is the number of training epochs. The y-axis is the top-1 error of 3,000 random val samples, evaluated on the center crop. We use ReLU as the activation for both cases. Both our initialization (red) and “Xavier” (blue) [7] lead to convergence, but ours starts reducing error earlier.

